Atividade de Revisão — Árvores

Aluno: Natan Adriel de Almeida Rodrigues
Disciplina: Estrutura de Dados II
Professora: Kadidja Valéria
Atividade individual

Etapa 1 — Revisão bibliográfica

Conceitos básicos

- **Nó:** cada elemento da árvore. Guarda um valor e as ligações para os filhos.
- **Raiz:** o primeiro nó, lá em cima. É o único que não tem pai.
- **Pai e filho:** se um nó A aponta para um nó B, A é pai de B e B é filho de A.
- **Folha:** nó que não tem nenhum filho.
- **Altura:** quantidade de níveis (arestas) do caminho mais longo da raiz até uma folha.
- **Percurso:** a ordem em que a gente visita os nós. Os principais são pré-ordem (raiz, esquerda, direita), em-ordem (esquerda, raiz, direita) e pós-ordem (esquerda, direita, raiz). Numa ABB, a em-ordem mostra os valores em ordem crescente.

Tipos de árvore

Árvore geral: cada nó pode ter quantos filhos quiser. Não tem regra de ordem. Serve para representar hierarquias, como pastas do computador ou o organograma de uma empresa.

Árvore binária: cada nó tem no máximo 2 filhos, um à esquerda e um à direita. É a base de várias outras árvores (ABB, AVL, rubro-negra e heap).

Árvore binária de busca (ABB): é uma árvore binária com uma regra: o que é menor que o nó fica à esquerda e o que é maior fica à direita. Para buscar ou inserir, começa na raiz e vai descendo, comparando. Se ela estiver equilibrada, a busca é rápida (O(log n)). O problema é que, se os valores entrarem já ordenados (1, 2, 3, 4...), ela vira quase uma lista e a busca fica O(n).

Árvore AVL: é uma ABB que se mantém balanceada. Em cada nó, a diferença de altura entre o lado esquerdo e o direito (fator de balanceamento) só pode ser -1, 0 ou 1. Quando uma inserção ou remoção quebra essa regra, a árvore faz rotações (simples ou duplas) para corrigir. Assim a busca fica sempre O(log n). É boa quando o programa faz muita busca.

Árvore rubro-negra: também é uma ABB balanceada, mas usa cores. Cada nó é vermelho ou preto, a raiz é preta, um nó vermelho não pode ter filho vermelho e todo caminho da raiz até as folhas tem a mesma quantidade de nós pretos. Quando uma regra é quebrada, ela troca cores e faz rotações. Ela é um pouco menos balanceada que a AVL, mas faz menos rotações, então é boa quando tem muita inserção e remoção. É usada em bibliotecas de linguagens, como o TreeMap do Java.

Árvore B: cada nó pode guardar várias chaves e ter vários filhos, e todas as folhas ficam no mesmo nível. Quando um nó enche, ele é dividido em dois e a chave do meio sobe para o pai. Como cada nó guarda muita coisa, a árvore fica baixa e precisa de poucas leituras no disco. Por isso é usada em bancos de dados e sistemas de arquivos.

Árvore B+: parecida com a B, mas os dados ficam só nas folhas. Os nós de cima guardam só chaves para guiar a busca. As folhas são ligadas uma na outra, como uma lista, então dá para ler um intervalo de valores em sequência (por exemplo, "todos os alunos com matrícula entre 100 e 200"). É muito usada em índices de banco de dados.

Heap: é uma árvore binária completa, normalmente guardada em um vetor. No max-heap, o pai é sempre maior ou igual aos filhos, então o maior valor fica na raiz. No min-heap é o contrário. Quando insere ou remove, o elemento sobe ou desce trocando de lugar até a regra voltar a valer. É usado em fila de prioridade e no heapsort.

Trie (árvore de prefixos): cada nível representa uma letra da palavra. Palavras que começam igual dividem os mesmos nós no começo. Por exemplo, "casa" e "carro" usam os mesmos nós para "c" e "a". A busca depende do tamanho da palavra, não de quantas palavras tem guardadas. É usada em autocompletar e corretor ortográfico.

Etapa 2 — Quadro comparativo

Estrutura	Como os dados ficam organizados	Regra principal	Operação ou ajuste importante	Exemplo de uso	Referência
Árvore geral	Nós com qualquer número de filhos	Não tem regra de ordem	Percorrer a hierarquia	Pastas do computador	Material de aula
Árvore binária	Nós com até 2 filhos	No máximo 2 filhos por nó	Percursos pré, em e pós-ordem	Expressões matemáticas	Material de aula
ABB	Árvore binária ordenada	Menor à esquerda, maior à direita	Busca comparando e descendo; não se balanceia	Busca simples de números	Cormen, cap. 12
AVL	ABB com fator de balanceamento	Diferença de altura entre os lados no máximo 1	Rotações simples e duplas	Sistemas com muita busca	Material de aula
Rubro-negra	ABB com nós vermelhos e pretos	Mesmo número de nós pretos em todo caminho	Recolorir e rotacionar	TreeMap do Java	Cormen, cap. 13
B	Nós com várias chaves e vários filhos	Todas as folhas no mesmo nível	Dividir o nó cheio (split)	Banco de dados, sistema de arquivos	Cormen, cap. 18
B+	Chaves de guia em cima, dados só nas folhas	Folhas ligadas entre si	Split e consulta por intervalo	Índices de banco de dados	Material de aula
Heap	Árvore binária completa em vetor	Pai maior (max) ou menor (min) que os filhos	Subir/descer o elemento (heapify)	Fila de prioridade, heapsort	Cormen, cap. 6
Trie	Um nível por letra	Palavras com o mesmo começo dividem os nós	Busca letra por letra	Autocompletar	Material de aula

Etapa 3 — Analogias

1. Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.

- **Estrutura:** Árvore AVL.
- **Por quê:** a AVL confere a altura dos dois lados de cada nó. Se um lado passa do outro por mais de 1, ela faz rotação para equilibrar.
- **Limite da analogia:** numa estante de verdade você teria que mexer em vários itens. Na AVL a rotação só troca algumas ligações entre nós, sem mover o resto.

2. Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida.

- **Estrutura:** Árvore B.
- **Por quê:** cada nó da árvore B é como uma página com várias chaves. Quando enche, ele é dividido em dois e a chave do meio sobe para o nó de cima.
- **Limite da analogia:** no catálogo a página só é dividida. Na árvore B, a chave do meio também sobe para o pai, e isso pode fazer o pai dividir também.

3. Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro.

- **Estrutura:** Heap (max-heap).
- **Por quê:** no max-heap o maior valor sempre fica na raiz, então é só pegar o de cima.
- **Limite da analogia:** numa fila normal todo mundo anda um passo. No heap, quando tira o topo, o último elemento vai para cima e vai descendo até achar o lugar certo. O resto não fica totalmente ordenado.

4. Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo.

- **Estrutura:** Trie.
- **Por quê:** na trie cada nível é uma letra, e palavras com o mesmo começo usam os mesmos nós.
- **Limite da analogia:** num índice de livro cada palavra aparece escrita inteira. Na trie o começo igual é guardado uma vez só.

5. Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.

- **Estrutura:** Árvore rubro-negra.
- **Por quê:** ela marca os nós como vermelho ou preto e, quando alguma regra de cor é quebrada, recolore e rotaciona. Assim nenhum caminho fica com mais que o dobro do tamanho de outro.
- **Limite da analogia:** as "cores" não são cores de verdade. No programa é só uma informação guardada em cada nó (tipo 0 ou 1).

6. Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo.

- **Estrutura:** Árvore B+.
- **Por quê:** os nós de cima só guiam a busca, os dados ficam nas folhas, e as folhas estão ligadas em sequência.
- **Limite da analogia:** num índice de papel, para ver o próximo assunto você volta ao índice. Na B+ você só passa para a folha do lado.

7. Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.

- **Estrutura:** Árvore binária de busca (ABB).
- **Por quê:** essa é exatamente a regra da ABB: menor vai para a esquerda e maior para a direita.
- **Limite da analogia:** a ABB não se arruma sozinha. Se os números entrarem em ordem, ela fica torta, parecendo uma lista, e a busca fica lenta.

Referências

- CORMEN, T. H. et al. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.
- Material de aula da disciplina Estrutura de Dados II (UDF), Profa. Kadidja Valéria.